

Team 075: Project Report

1. Project Direction

Our initial proposal was to build a relational Personal Health Record (PHR) system to deal with fragmented medical data and allow structured querying of patient records.

The final project stayed very close to this idea. We built a centralized relational database that organizes patient data such as encounters, lab results, medications, and conditions into a clear and structured schema.

The system includes core tables like Patient, Encounter, LabResult, PatientMedication, and PatientCondition, along with supporting lookup tables. These are connected using appropriate keys, relationships, and constraints.

Overall, the project remained consistent with our original direction, focusing on relational modeling, data organization, and SQL-based querying.

2. Usefulness

We believe our application successfully achieved its goal of improving access to and exploration of medical data.

Medical data is often spread across multiple disconnected systems, making it difficult for users to fully understand their records. Our application addresses this by providing:

- Centralized record storage
- Cross-table relational querying
- Aggregated summaries
- Keyword-based search across diagnoses and medications
- Strong database integrity through constraint enforcement

While existing systems like Epic MyChart allow users to view their data, they do not support structured querying or relational exploration at the SQL level.

One limitation is that the system is restricted by the NHANES dataset, which limits the variety of conditions and medical relationships available.

Overall, our system meets its main goals of improving transparency, enabling organized queries, and maintaining strong database integrity.

3. Schema / Data Source Changes

We made a change to our data source during the project.

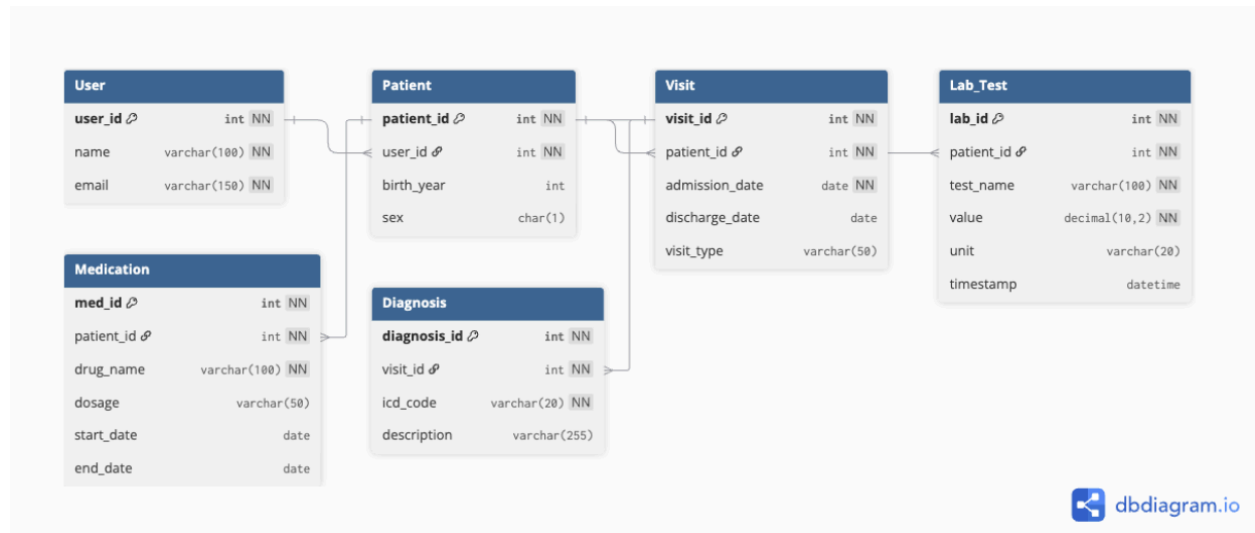
Originally, we planned to use the MIMIC-IV clinical dataset along with CDC datasets. However, we were unable to use MIMIC-IV due to access restrictions and data use limitations, as it requires special authorization and is intended for formal research use.

Instead, we used publicly available health data from the Centers for Disease Control and Prevention, specifically the NHANES dataset. This dataset provides anonymized patient records and allowed us to work with realistic medical data while staying within project constraints.

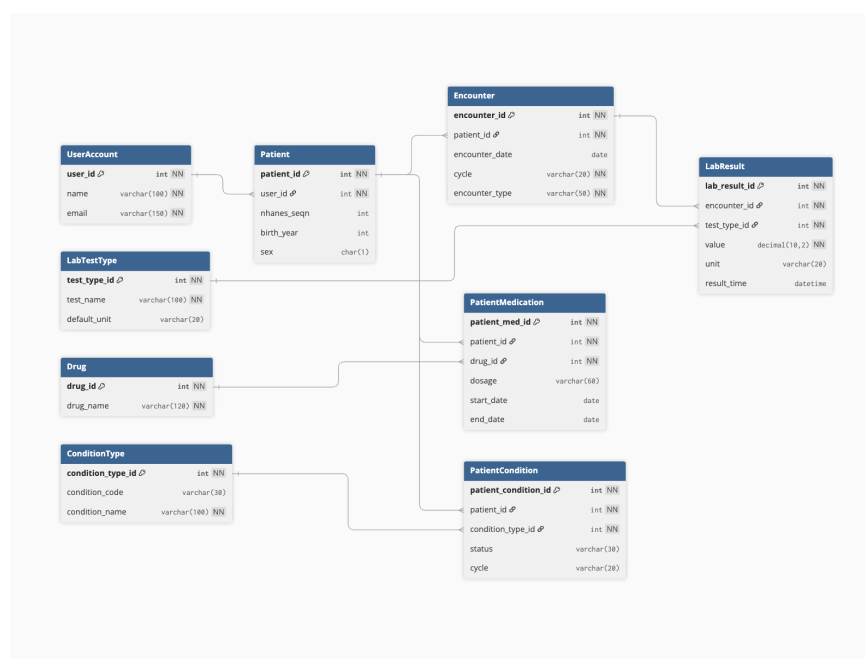
Our schema design remained aligned with our original plan, but was adapted to fit the structure of the NHANES data. We will outline the changes made to the tables and schema below when we speak of the changes to our ER Diagram and table implementations.

4. Changes to ER Diagram & Table Implementations

4.1. Our Original ER Diagram:



4.2. Our Implemented ER Diagram



We will not paste all our table implementations below as that would take up unnecessary space, and those implementations follow on from the ER diagrams, instead we will outline the differences below.

Differences in our ER Diagram and consequent table implementations occurred as a result of using the CDC NHANES dataset instead of the originally proposed MIMIC-IV dataset. A description of the changes made is as follows:

- User was renamed to UserAccount to avoid conflicts with reserved SQL keywords.
- Visit was renamed to Encounter, which better matched the medical record terminology used in the final system.
- Patient now includes `nhanes_seqn`, which preserves the original NHANES identifier and helps connect our normalized data back to the source data.
- Lab_Test was split into LabTestType and LabResult. This avoids repeating lab test names and units for every result.
- Medication was split into Drug and PatientMedication. This avoids repeating drug names and makes medication records more normalized.
- Diagnosis/Condition was redesigned into ConditionType and PatientCondition. This separates condition definitions from each patient's condition records.
- LabResult now connects to Encounter instead of directly to Patient, which better represents when the lab result occurred.
- The final schema adds stronger constraints, including unique keys, foreign keys with update/delete rules, and date checks for medications.

The implemented design is more suitable because it is more normalized and better matches the structure of the NHANES data. By separating lookup information, such as lab test names, drug names, and condition types, from patient-specific records, the schema reduces redundancy and makes updates easier. It also gives the database stronger integrity through clearer relationships and constraints. Overall, the final design is a better fit for storing realistic health records and supporting SQL-based querying.

5. Changes in Functionality

Compared to our original proposal, we narrowed some of the planned functionality to better fit the dataset and application scope.

Implemented functionality:

- Medication CRUD, including add, edit, delete, and discontinue
- Read-only patient history, including labs, encounters, conditions, and medications

- Patient search by name, medication, condition, sex, and birth year
- Analytics overview with aggregate summaries

Removed or limited functionality:

- Adding/deleting patients, visits, lab records, and diagnoses
- Updating lab values or visit metadata
- Full lab test name search and date range filtering

The main reason for these changes was that NHANES is a fixed public dataset. Because of this, modifying core patient, lab, and diagnosis records would not be realistic. We treated these records as historical data that should not be changed. Instead, we focused write operations on medications, where create, update, delete, and discontinue actions make more sense for a user.

Overall, we kept the most useful parts of the original functionality while making the final application more realistic and manageable.

6. Advanced Database Programs

Our advanced database programs mainly support the medication workflow, since that is the part of the application where users can actually make changes.

The triggers (on insert and update) make sure medication dates are always valid. If a user enters an `end_date` before the `start_date`, the trigger fixes it and logs the issue. This helps prevent bad data from getting into the database and avoids confusing errors for the user.

The stored procedure handles the “discontinue medication” action. It checks if the record exists, updates the `end_date`, and logs what happened. This is done inside a transaction, so both the update and the log either happen together or not at all. This keeps the data consistent.

Overall, these programs help keep the database clean and reliable. They act as a safety layer, so even if something is missed in the application, the data is still correct.

7. Technical Challenges Encountered

7.1. Pranay

The main challenge was loading the NHANES dataset into our normalized schema. The data is split across multiple independent files that only share a common key (SEQN), and many required fields in our schema (such as birth_year or foreign keys) had to be derived or joined across files. Directly inserting this data into production tables caused foreign key violations and constraint failures due to missing or out-of-order data.

To solve this, we introduced a staging layer with tables that mirror the raw data and have no constraints. We first loaded all raw data into these staging tables, and then performed a second transformation step using INSERT ... SELECT queries to clean, join, and derive the required fields before inserting into the final schema.

Lesson: When working with real-world datasets and a normalized schema, use a staging layer to separate raw data loading from transformation and constraint enforcement.

7.2. Krishna

The main challenge was designing queries that were both meaningful from a medical perspective and technically complex. Many useful queries required joining multiple tables (e.g., Patient → Encounter → LabResult → LabTestType) along with aggregation and filtering.

Some queries also required subqueries that could not be replaced with joins. For example, comparing a patient's cholesterol level to the population average required computing a global aggregate in a separate subquery. Similarly, identifying patients with multiple conditions required careful use of GROUP BY and HAVING, since filtering too early would miss valid results.

Lesson: Query design requires careful planning of when to use joins, aggregation, and subqueries. Avoid filtering before aggregation, and use subqueries when comparing against global values.

7.3. Shankar

The main challenge was building a natural language query interface without using an LLM. This required designing a rule-based system that maps user input into structured query parameters, which are then converted into SQL.

The difficulty was handling ambiguous or unsupported queries. Some inputs did not clearly map to known metrics or fields, while others could lead to incorrect interpretations if forced into a query. To address this, we restricted the system to a defined set of supported terms and added fallback responses that suggest valid query formats when the input cannot be parsed.

Lesson: In rule-based systems, it is better to clearly reject unsupported queries than return incorrect or misleading results.

7.4. Akshaj

The main challenge was deciding which indexes actually improve performance for our queries. Since our workload includes joins, aggregations, and subqueries, it was not clear which columns or combinations to index.

We tested different indexing strategies using EXPLAIN ANALYZE, including indexes on foreign keys, filter columns, and composite indexes. We found that some indexes improved join performance, while others were ignored or even slowed queries. A key issue was that foreign key columns are not automatically indexed in MySQL, which initially caused inefficient scans.

Lesson we learnt: Indexing should be based on actual query patterns and verified using query plans. Understanding how the optimizer uses indexes is essential.

8. Other Changes

There were a few changes other than the aforementioned ones in the report, but some to note are:

- Change in advanced feature focus: Advanced database programs were originally planned around visits, diagnoses, and lab validation, but were instead implemented around the medication workflow.
- Change in AI component design: The proposed RAG-based AI query system was replaced with a deterministic rule-based assistant without external models or embeddings.
- UI structure changes: Some planned sections (Dashboard, Visits, separate Search, standalone AI query) were removed or merged, while a Conditions view was added.

9. Future Work

Future improvements can focus on expanding the database and backend capabilities.

- Support a richer dataset with more conditions, lab tests, and medical relationships (maybe use MIMIC-IV)
- Enable controlled updates for core tables (patients, labs, encounters) with proper validation
- Add more advanced queries and analytics, such as longitudinal trends and cohort-based analysis
- Improve indexing and query optimization for larger datasets
- Extend the assistant to support more complex queries and broader medical concepts

10. Teamwork & Division of Labour

Our team maintained strong communication throughout the project using a group chat for quick updates and coordination, along with regular virtual calls to discuss progress and resolve issues. We ensured that work was distributed fairly across all members during all of the stages (You can see an approximate stagewise breakdown of responsibilities below). Overall, the workload was balanced, and we were able to coordinate effectively as a team.

Stage 1:

We equally split sub-sections of the proposal and had a meeting to discuss and effectively combine all our inputs to product the final projectproposal.md

Stage 2:

Similar to Stage 1, we all split sub-sections of the ER Diagram design and normalization, then collaborated to create the final erdiagram.md.

Stage 3:

Pranay and Krishna handled the data ingestion and staging. Akshaj and Pranay handled the advanced queries. Akshaj and Shankar then collaborated on the indexing strategy. Shankar and Krishna handled the indexing analysis.

Stage 4:

Pranay handled most of the initial frontend creation, with Krishna assisting in the refinement. Pranay and Akshaj collaborated on the advanced database features. Shankar and Krishna collaborated on the creative component. The project report was primarily drafted by Akshaj and Shankar, and everyone contributed to the refinement. The project video was a combined effort of all the team members.

These responsibilities were approximate, and there was a lot of flexibility and fluidity in the manner in which we collaborated on the stages.